



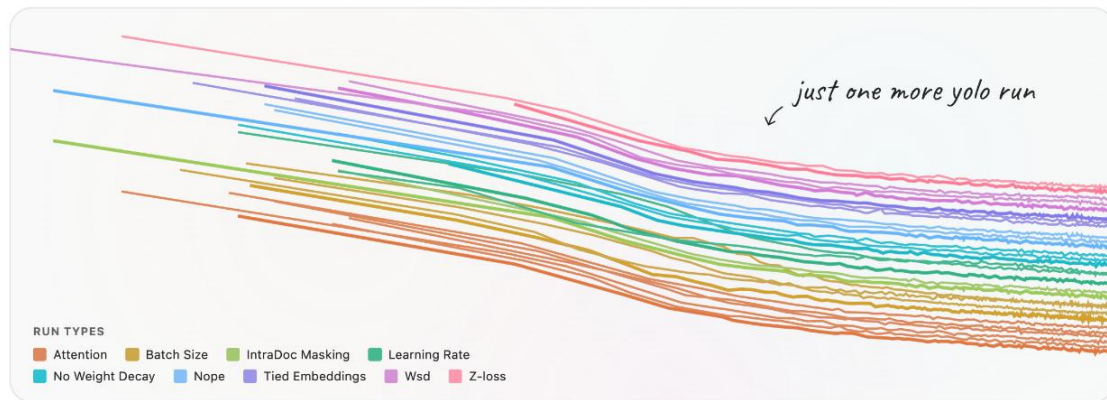
The Smol Training Handbook

Lessons from Training State-of-the-Art LLMs

Nouamane Tazi



The Smol Training Playbook: The Secrets to Building World-Class LLMs



*A practical journey through the challenges, decisions, and messy reality
behind training state-of-the-art language models*

AUTHORS

[Loubna Ben Allal](#), [Lewis Tunstall](#), [Nouamane Tazi](#),
[Elie Bakouch](#), [Ed Beeching](#), [Carlos Miguel Patiño](#),
[Clémentine Fourier](#), [Thibaud Frere](#), [Anton Lozhkov](#),
[Colin Raffel](#), [Leandro von Werra](#), [Thomas Wolf](#)

AFFILIATION

[Hugging Face](#)

PUBLISHED

Oct. 30, 2025

PDF

[Download PDF](#)

Table of Contents

Introduction

[How to Read This Guide](#)

[Training Compass: Why → What → How](#)

Introduction

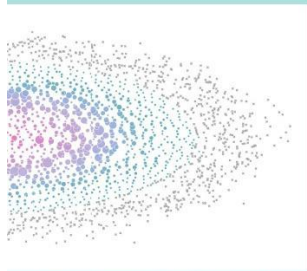
What does it actually take to train a high-performance LLM today?

Published research makes it look straightforward: strategic architecture choices, carefully

Reading time: 2-4 days.

The Ultra-Scale Playbook: Training LLMs on GPU Clusters

The Ultra-Scale Playbook: Training LLMs on GPU Clusters



NOUAMANE TAZI, FERDINAND MOM, HAOJUN ZHAO, PHUC NGUYEN,
MOHAMED MEKKOURI, LEANDRO VON WERRA, THOMAS WOLF 2025

We ran over 4,000 scaling experiments on up to 512 GPUs and measured throughput (size of markers) and GPU utilization (color of markers). Note that both are normalized per model size in this visualization.

[ORDER BOOK](#)[GET PDF](#)

AUTHORS

Nouamane Tazi, Ferdinand Mom, Haojun Zhao, Phuc Nguyen,
Mohamed Mekouri, Leandro Werra, Thomas Wolf

AFFILIATION

Hugging Face

PUBLISHED

Feb 19, 2025

Table of Contents ▼

High-Level Overview

First Steps: Training on One GPU

- Memory usage in transformers
- Profiling the memory usage
- Memory for weights/grads/optimizer states
- Memory for activations
- Activation recomputation
- Gradient accumulation
- Profiling GPU compute and communication

Data Parallelism

- First optimization: Overlap gradient synchronization with backward pass
- Second optimization: Bucketing gradients
- Third optimization: Interleave with model execution

Thousands of GPUs humming in perfect harmony. That's what it takes to train today's most powerful AI models – a symphony of computing power that until recently was the exclusive domain of elite research labs. Open source has transformed this landscape, but not completely. Yes, you can download the latest Llama or DeepSeek models. Yes, you can read their technical and experiment reports. But the most challenging part – the training code, the knowledge and techniques necessary to coordinate GPUs to train these massive systems – remains shrouded in complexity and spread around in a series of disconnected papers and often private codebases.

This open source book is here to change that. Starting from the basics, we'll walk you through the knowledge necessary to scale the training of large language models (LLMs) from one GPU to tens, hundreds, and even thousands of GPUs, illustrating theory with practical code examples

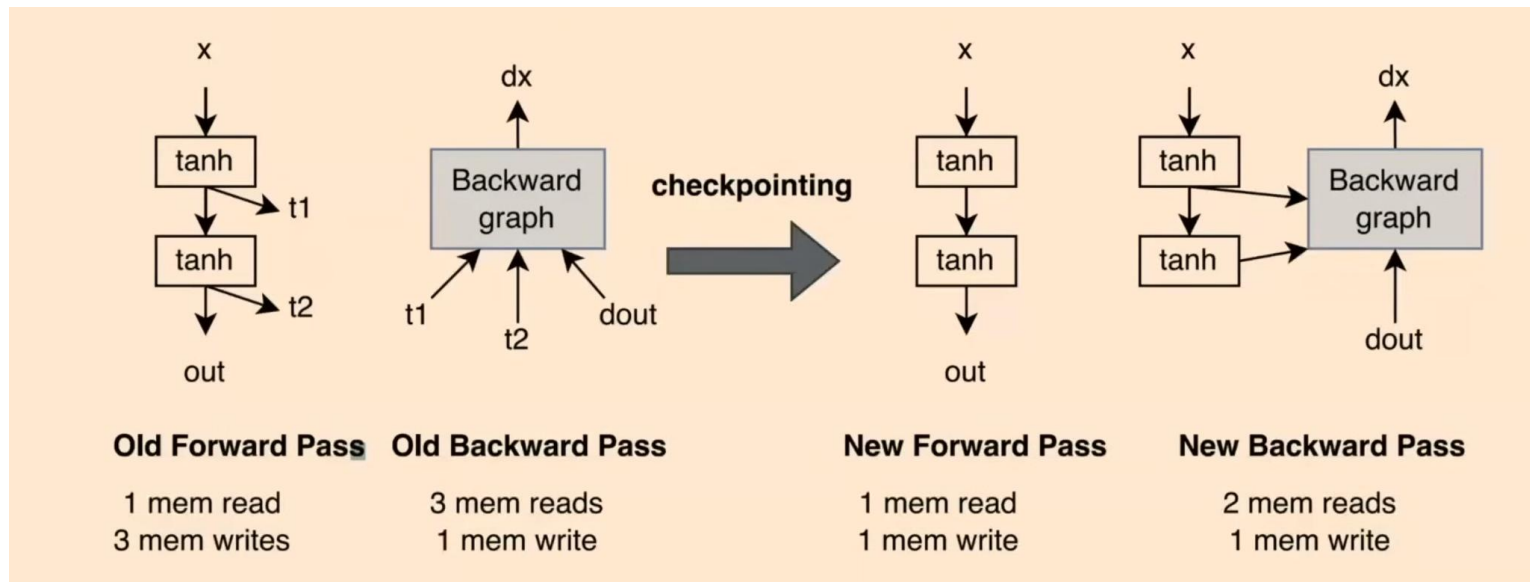
Reading time: 2-4 days.
For the best reading experience, we recommend not using a mobile phone.

Key lesson for today

Whether on 1 or thousands of GPUs, there's always a tradeoff between communication and compute under memory constraint

.. let's start with 1 GPU

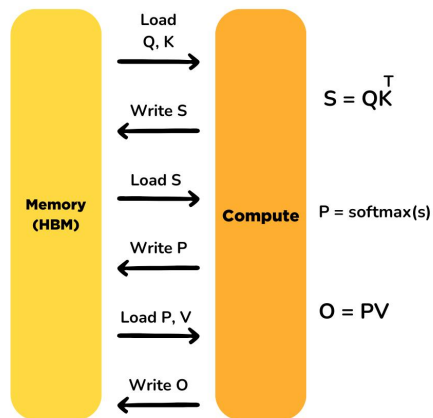
On 1 GPU: Activation Recomputation



Less memory, less comms -> more compute

On 1 GPU: Flash-Attention

Standard Attention Implementation

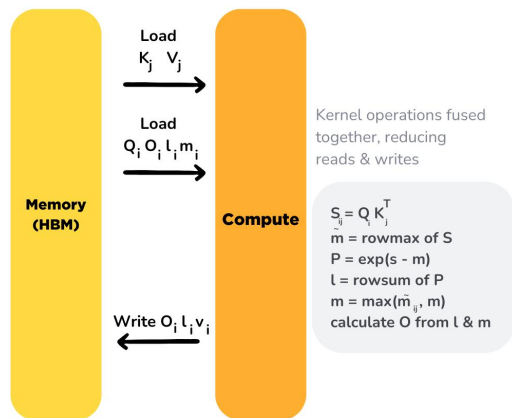


$$S = QK^T$$

$$P = \text{softmax}(s)$$

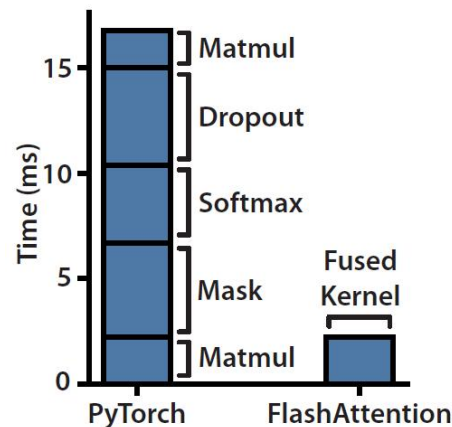
$$O = PV$$

Flash Attention



Initialize O, l and m matrices with zeroes. m and l are used to calculate cumulative softmax. Divide Q, K, V into blocks (due to SRAM's memory limits) and iterate over them, for i is row & j is column.

Attention on GPT-2



Less memory, less comms -> more compute

On 1 GPU: Roofline Model



source: [NVIDIA Nsight Compute Profiling Guide](#)

Limited by memory bandwidth

Limited by compute throughput

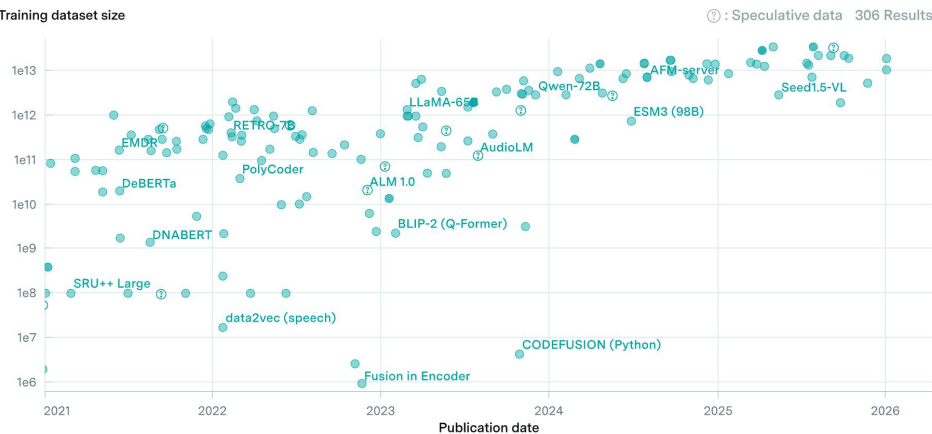
Roofline model shows kernel performance boundaries

Scaling training

tokens

Notable AI models

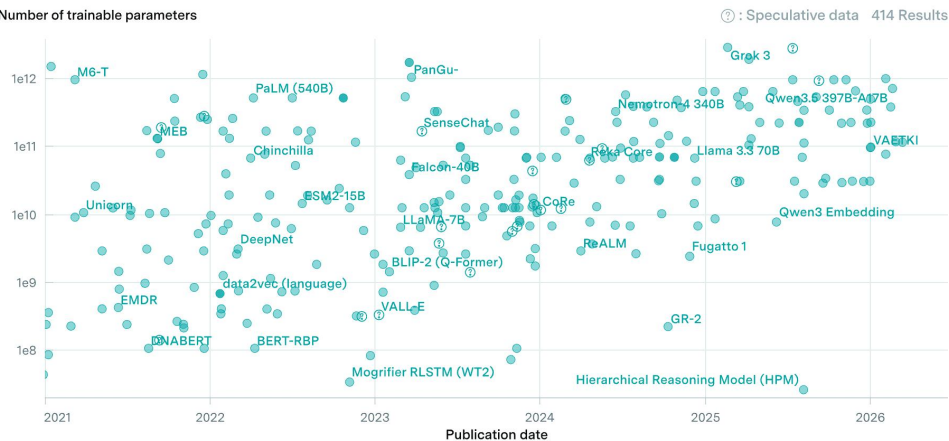
Training dataset size



EPOCH AI | CC-BY

epoch.ai

Number of trainable parameters



parameters

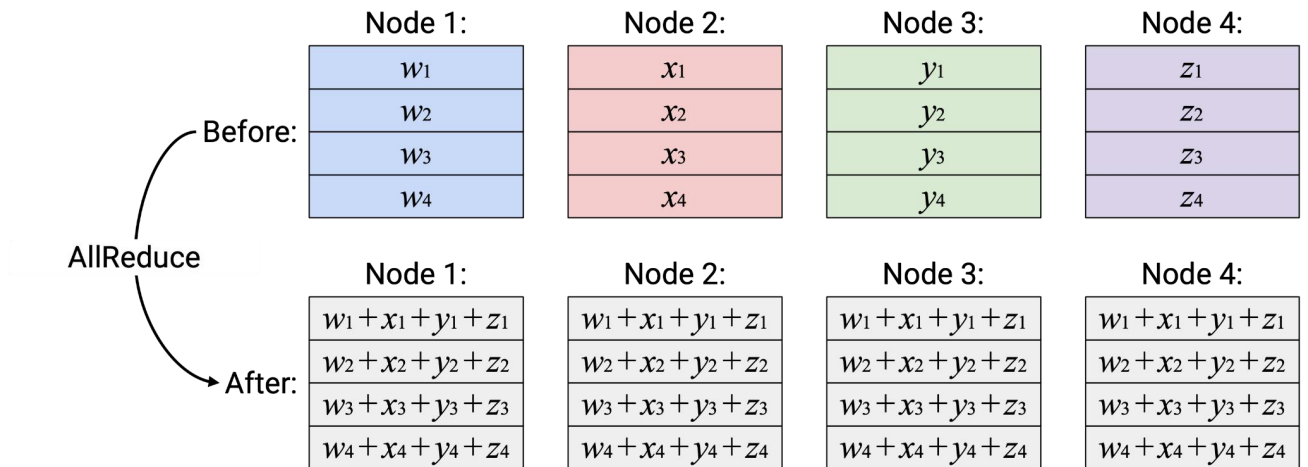
Scaling training

5 ways to scale training (can be combined)

1. Data Parallelism
2. Context Parallelism
3. Tensor Parallelism
4. Pipeline Parallelism
5. Expert Parallelism

AllReduce comm op

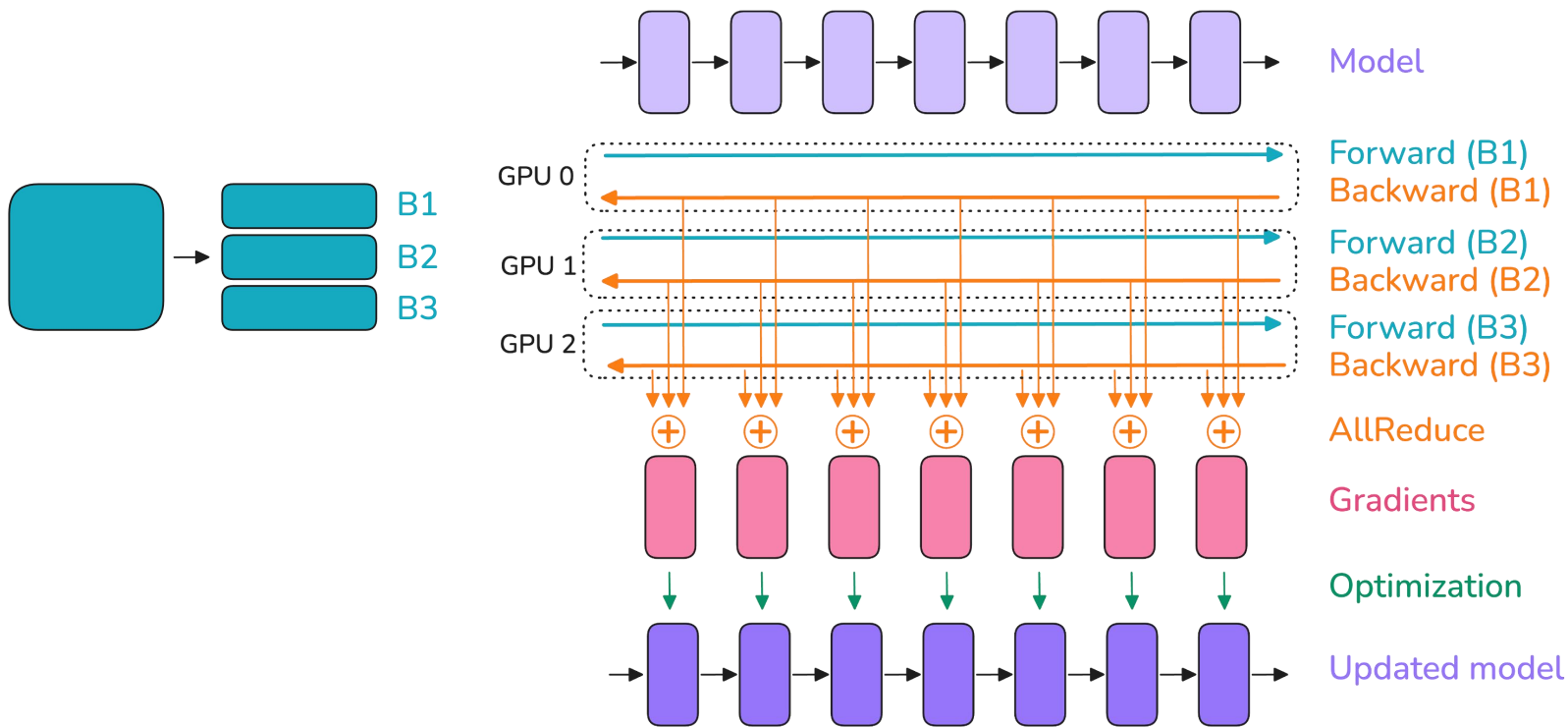
Example of a distributed communication operation between 4 GPUs



** Node = GPU*

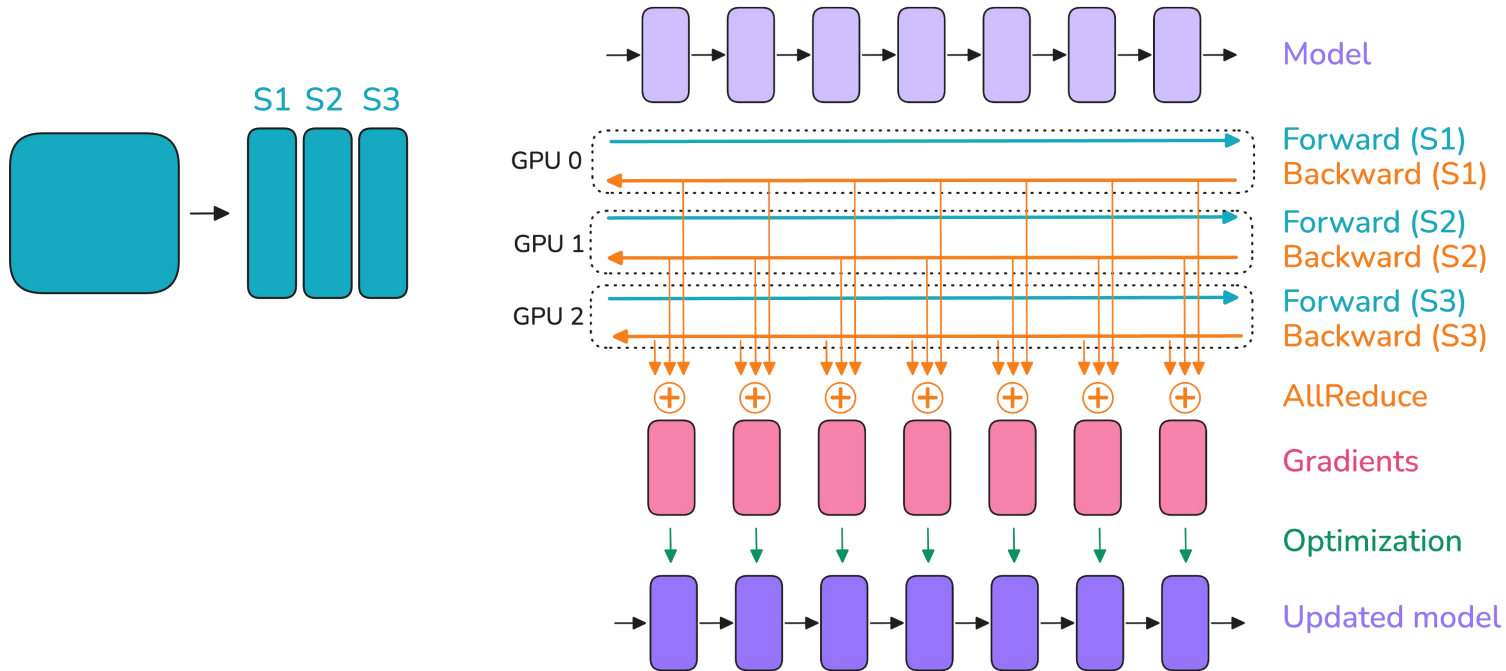
distributed sum

Fitting batch size: Data Parallelism



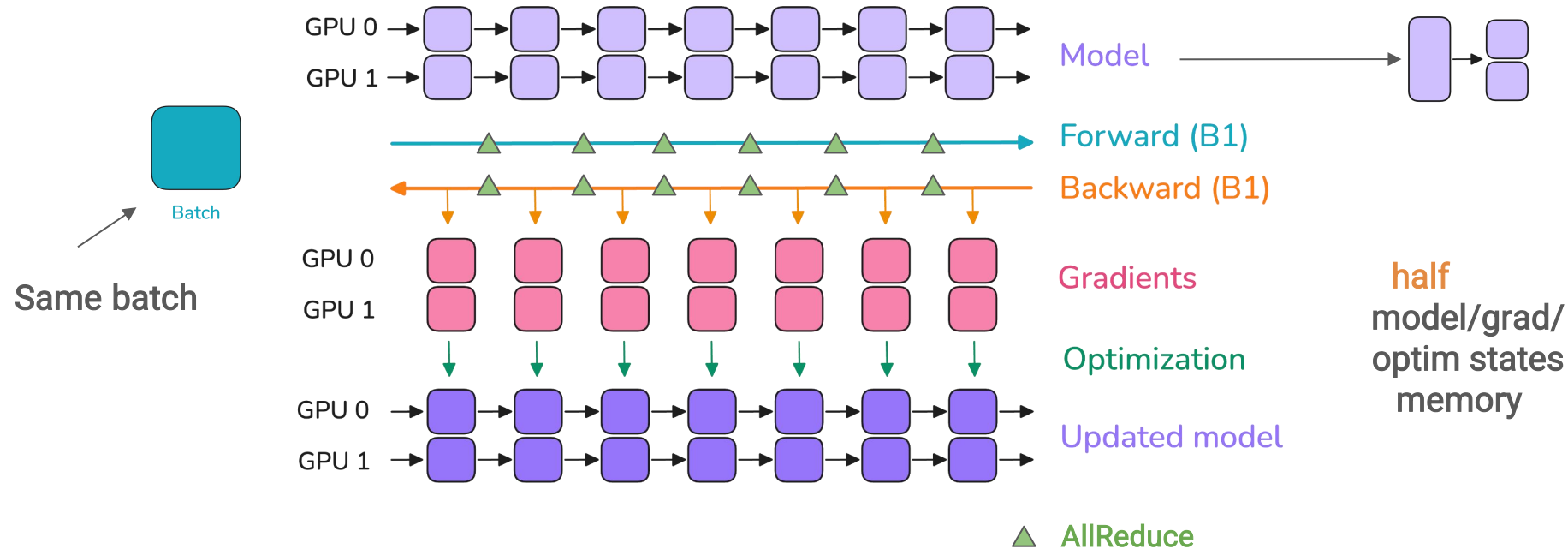
Less memory, less compute -> more comms

Fitting sequence length: Context Parallelism



Less memory, less compute -> more comms

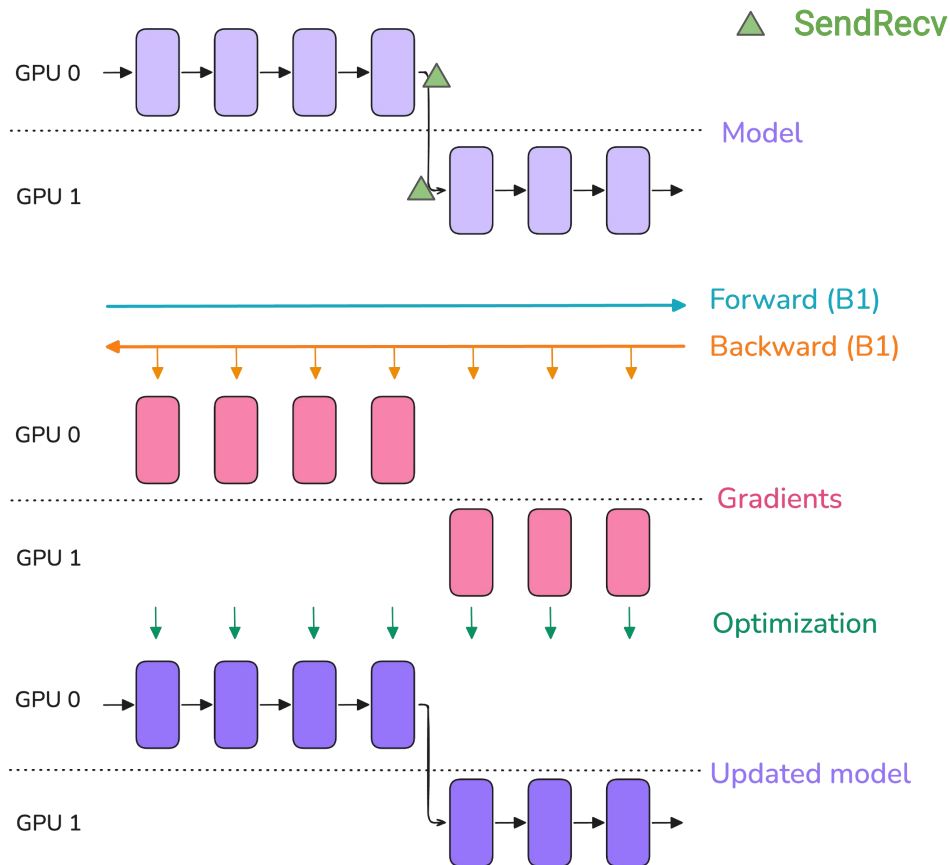
Fitting model: Tensor Parallelism



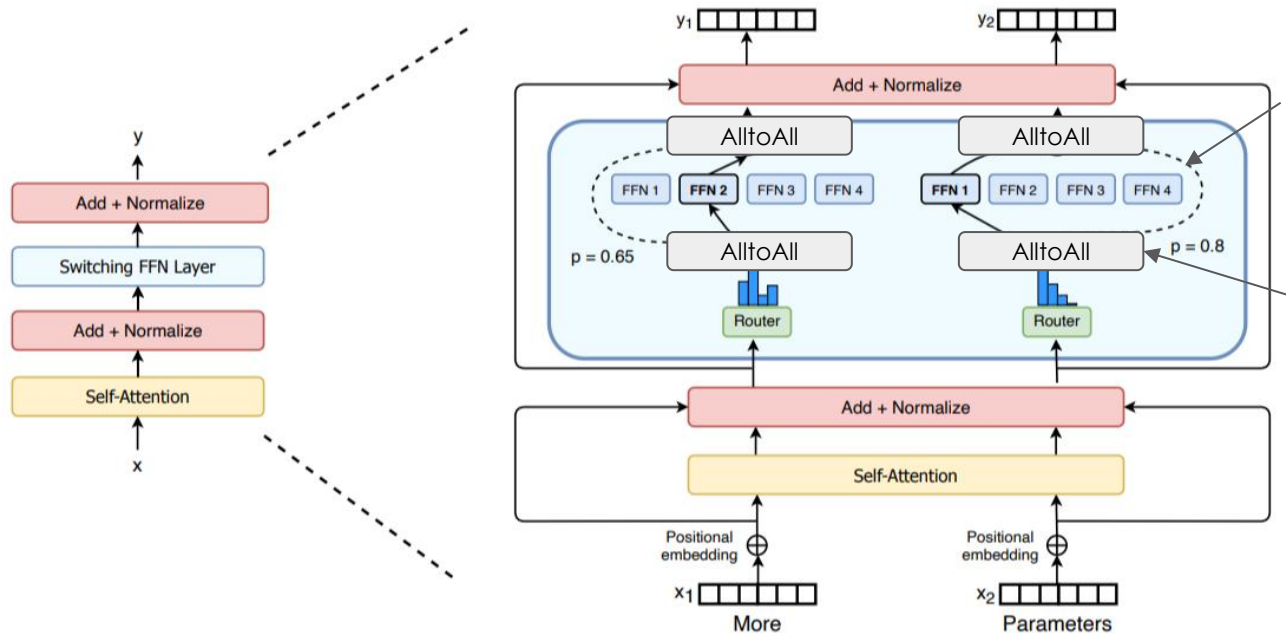
Less memory, less compute -> more comms

Fitting model: Pipeline Parallelism

Less memory, less compute
-> more comms

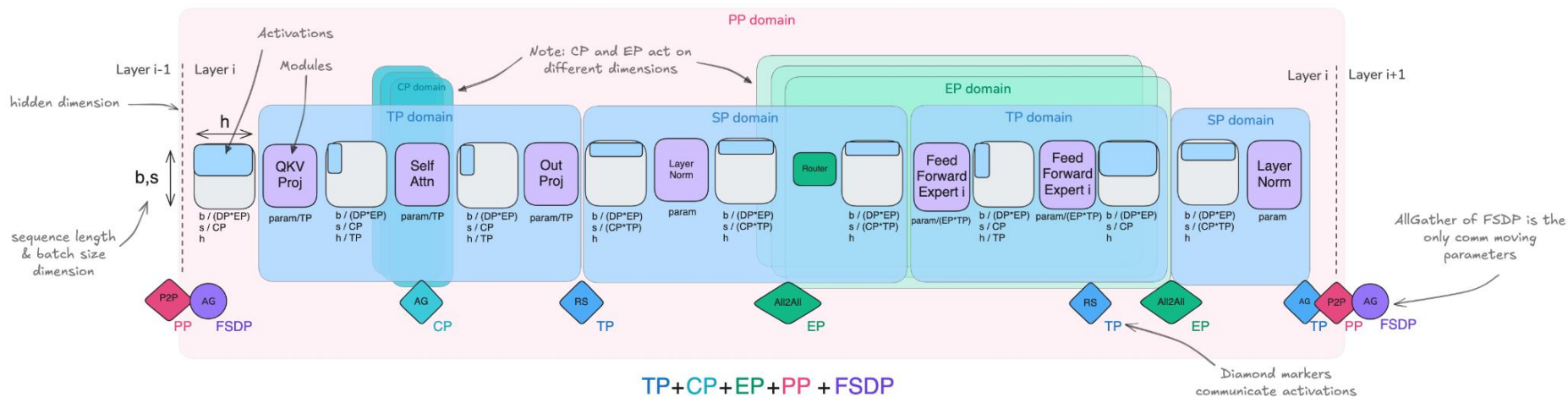


Fitting model: Expert Parallelism



Less memory, less compute -> more comms

Combining 5D parallelism



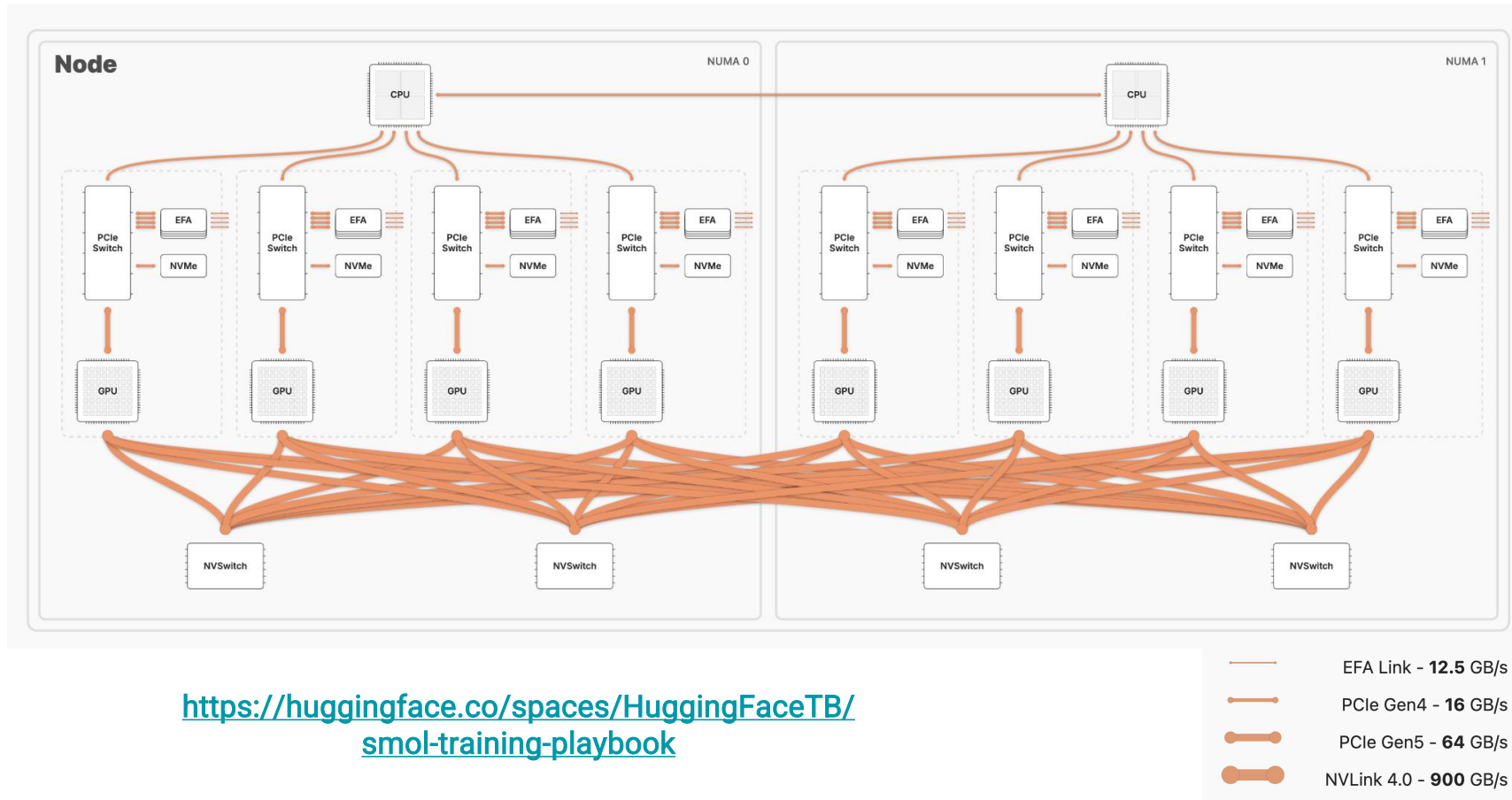
Different sharding strategies for different use cases

"Few" issues with scaling

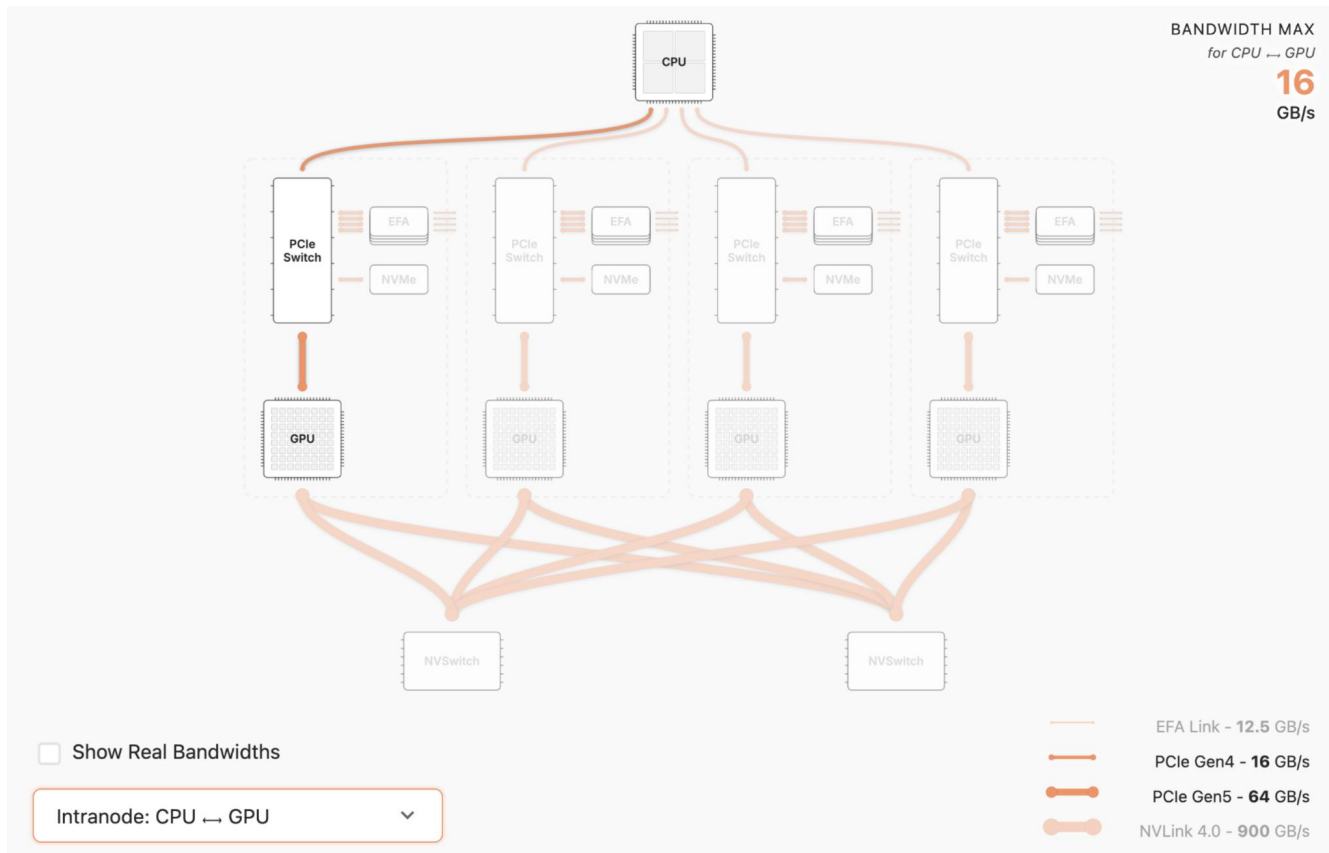
Failure Symptoms	Failure Domain			Likely Failure Cause
	User Program	System Software	Hardware Infra	
OOM	✓	✗	✗	User Bug
GPU Unavailable	✗	✓	✓	PCIe error, Driver/BIOS, thermals
GPU Memory Errors	✗	✗	✓	Thermal Noise, Cosmic Rays, HBM Defect or Wear
GPU Driver/Firmware Error	✗	✓	✗	Outdated Software, High Load
GPU NVLink Error	✗	✗	✓	Electro/Material Failure, Switch
Infiniband Link	✗	✗	✓	Electro/Material Failure, Switch
Filesystem Mounts	✗	✓	✗	Failed Frontend Network, Drivers in D State, Storage Backend
Main Memory Errors	✗	✗	✓	Circuit Wear, Thermal Noise, Cosmic Rays
Ethlink Errors	✗	✗	✓	Electro/Material Failure, Switch
PCIe Errors	✗	✗	✓	GPU Failure, Poor Electrical Contacts
NCCL Timeout	✓	✓	✓	Userspace Crash, Deadlock, Failed HW
System Services	✓	✓	✓	Userspace Interference, Software Bugs, Network Partition

TABLE I: Taxonomy of Failures. Users and cluster operators must infer a cause from a potentially ambiguous symptom. A common error is to misattribute the cause to the wrong component, especially when multiple domains are suspect.

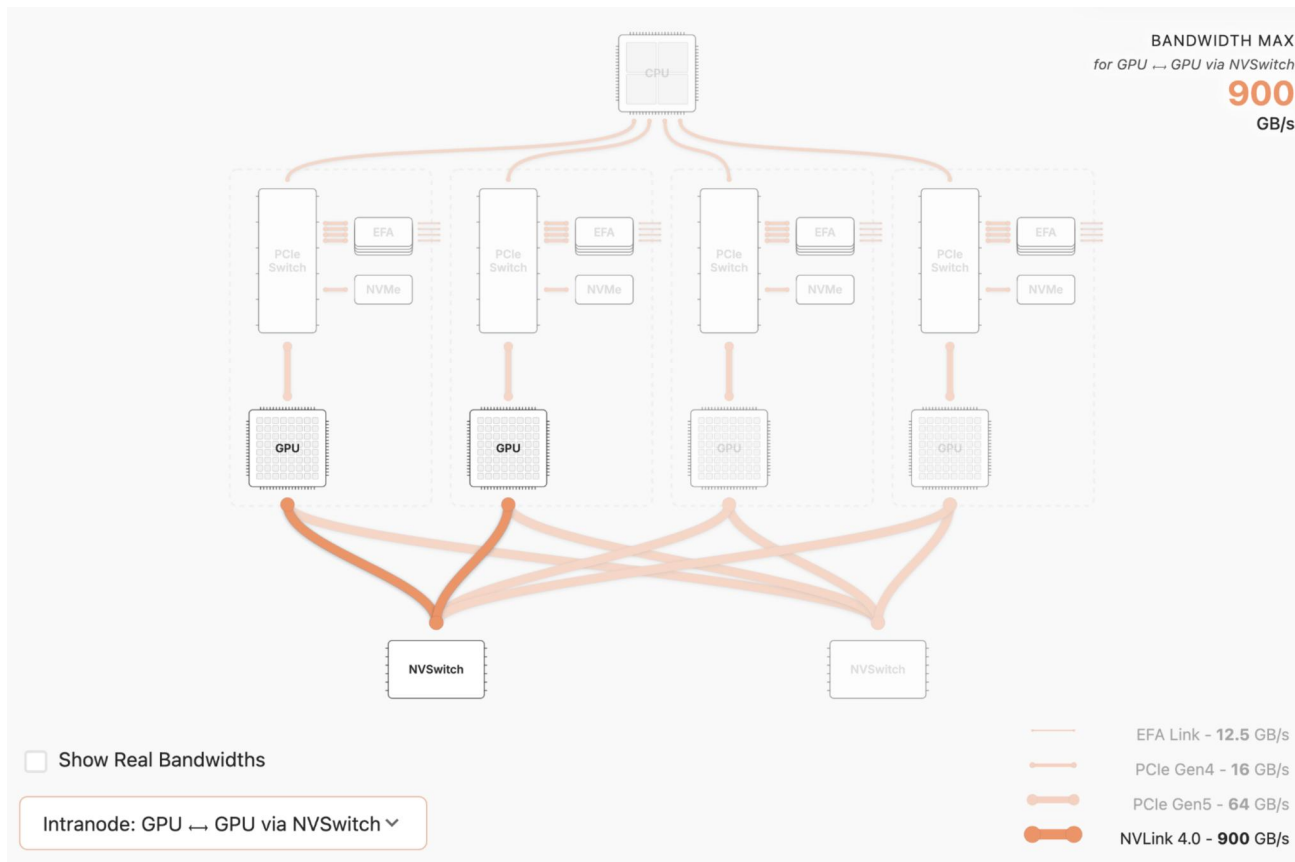
Understanding our infra



Understanding our infra

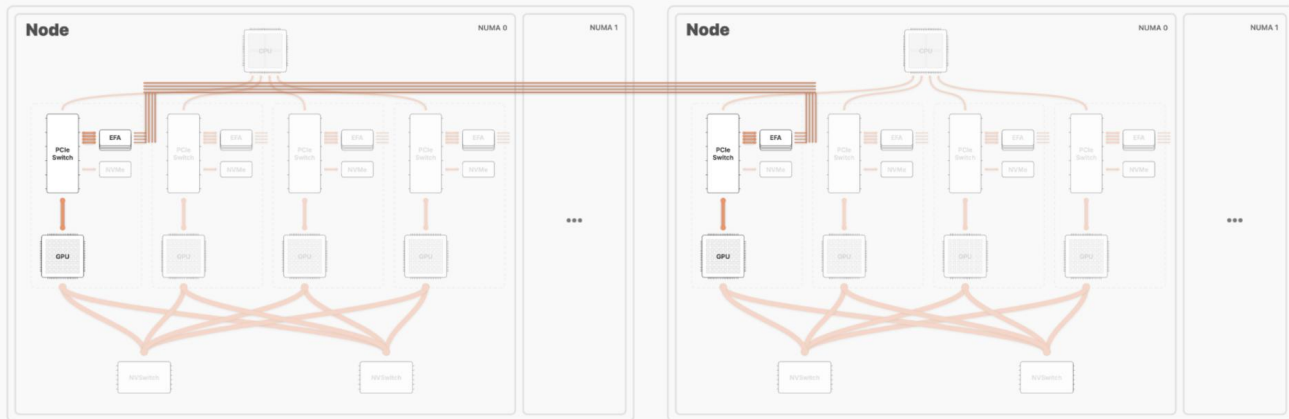


Understanding our infra



Understanding our infra

BANDWIDTH MAX
for GPU $\leftrightarrow$ GPU via EFA
50
GB/s

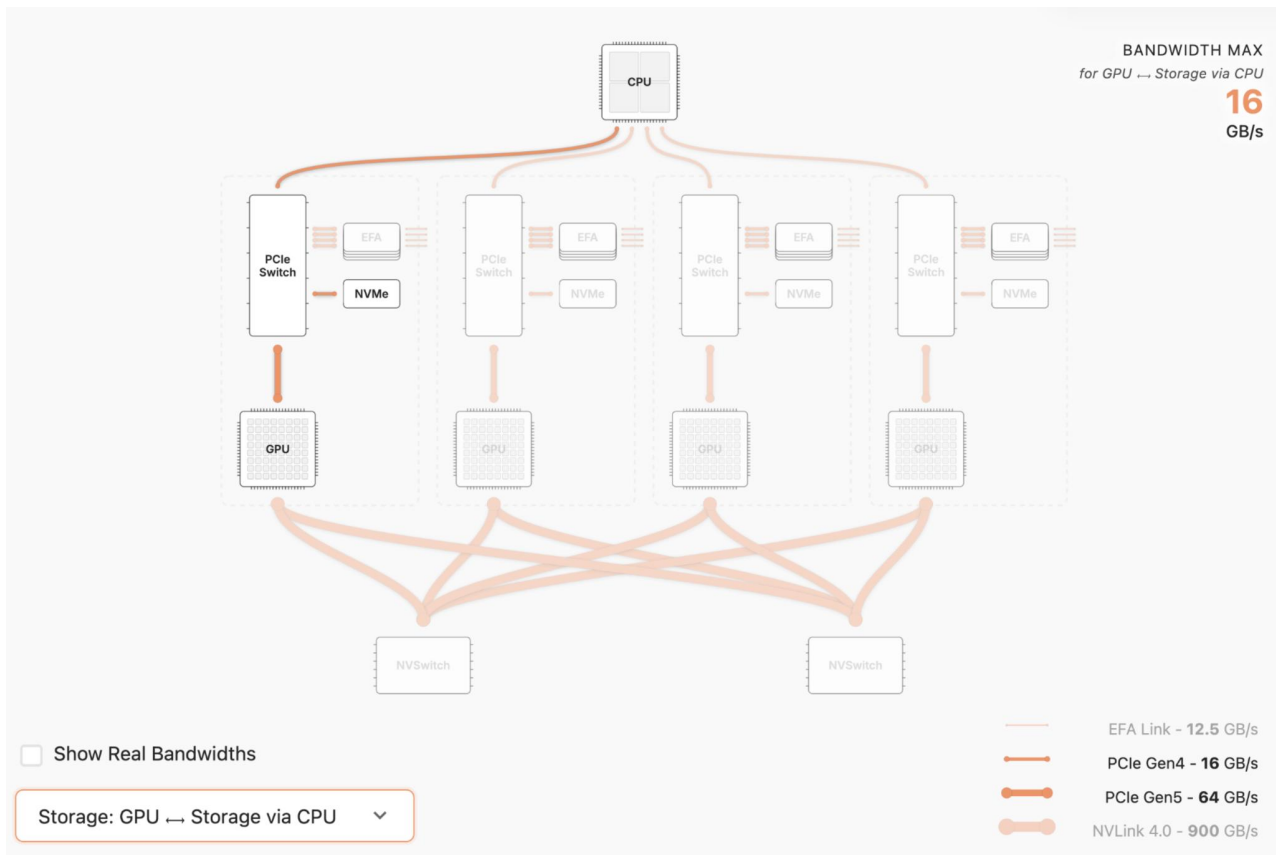


☐ Show Real Bandwidths

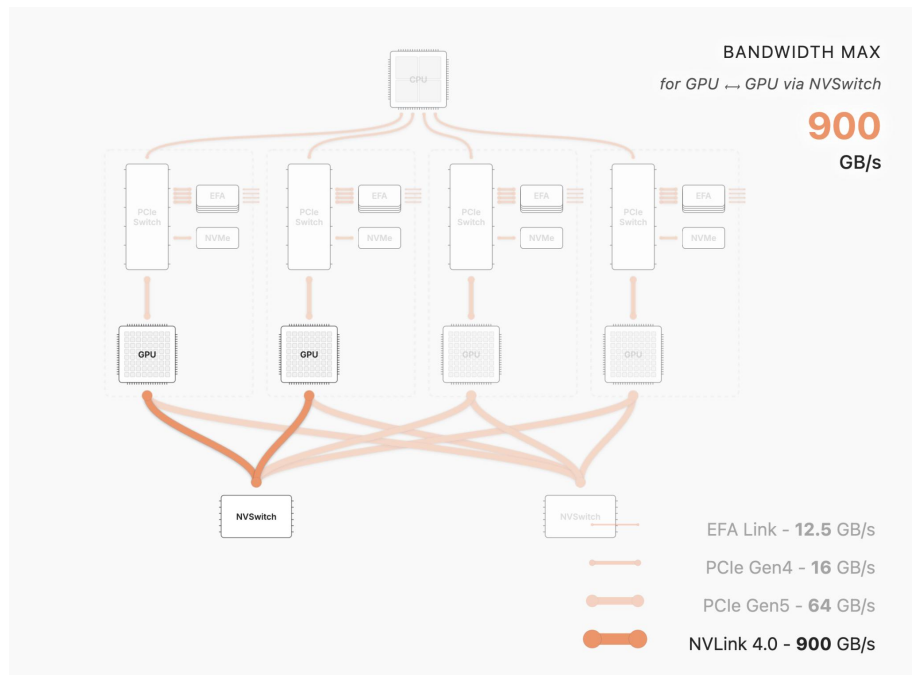
Internode: GPU $\leftrightarrow$ GPU via EFA

- EFA Link - 12.5 GB/s
- PCIe Gen4 - 16 GB/s
- PCIe Gen5 - 64 GB/s
- NVLink 4.0 - 900 GB/s

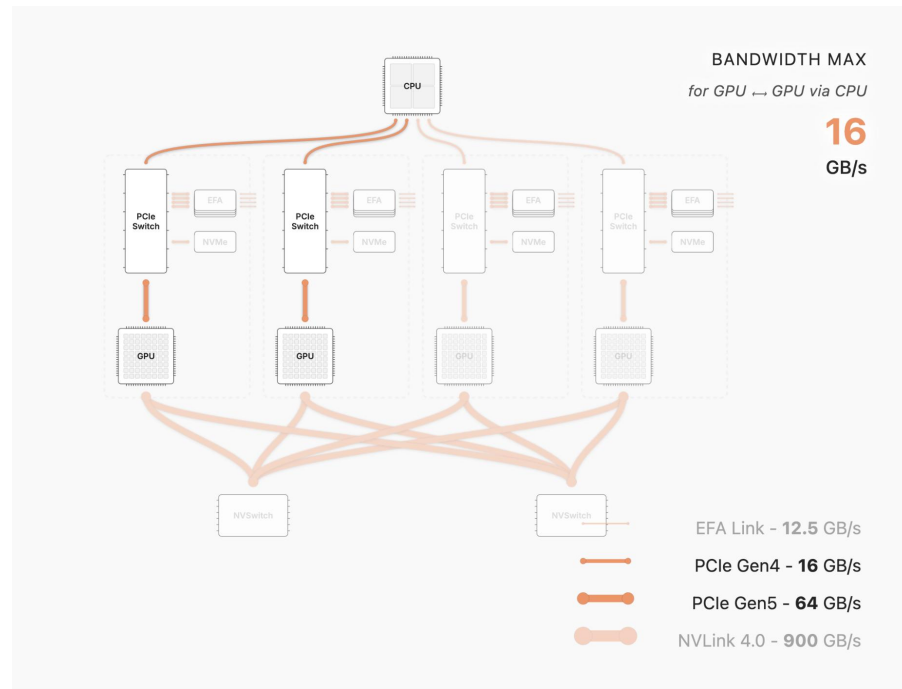
Understanding our infra



Some hard-earned lessons



default



FI_PROVIDER=efa NCCL_P2P_DISABLE=1

NCCL flags

Some hard-earned lessons

EFA Cheatsheet

[awslabs/awesome-distributed-training](https://aws-labs.github.io/awesome-distributed-training/)

1. Settings via environment variables

For optimized performance, you may need to set additional environment variables depending on the versions of your libfabric.

Setting	Explanation
<code>NCCL_DEBUG=info</code>	Set this to get debug information from NCCL, this will allow you to see if NCCL is using EFA and what versions it's using. This will print out a lot of debug information so we advise turning it off unless you suspect NCCL issues, see NCCL_DEBUG for more info.
<code>FI_EFA_USE_HUGE_PAGE=0</code>	Set to 0 when you see <code>os.fork()</code> causes <code>OSError: Cannot allocate memory</code> . Typically happen by multi-process PyTorch data loader. Disabling huge page causes minor performance hit, but it's needed to prevent fork fails due to the operating system running out of huge pages.
<code>FI_EFA_FORK_SAFE=1</code>	Not needed for kernel $\geq$ 5.15. Still fine to set it though no effect. See ref .
<code>FI_EFA_USE_DEVICE_RDMA=1</code>	Do not set for libfabric $\geq$ 1.18.0 and aws-ofi-nccl $\geq$ 1.7.0. It's not harmful to set it on p4/p5 on the newer software, but you just don't have to set it.
<code>FI_EFA_SET_CUDA_SYNC_MEMOPS=0</code>	Set this on efa-installer $<$ 1.29.1 and nccl $\geq$ 2.19.0 to prevent NCCL error <code>register_rail_mr_buffer:617 NCCL WARN NET/OFI Unable to register memory (type = 2) for device 4. RC: -22, Error: Invalid argument</code> .
<code>FI_EFA_ENABLE_SHM_TRANSFER=1</code>	Not needed. This is really a no-op, the default already to enable SHMEM
<code>FI_PROVIDER=efa</code>	Use for aws-ofi-nccl $\leq$ 1.5.0 AND p4/p5 instances.
<code>NCCL_PROTO=simple</code>	Use for aws-ofi-nccl $\leq$ 1.5.0 and p4/p5 instances.
<code>NCCL_SOCKET_NTHREADS</code>	Not applicable for EFA.

cheatsheets /
recommendations

Some hard-earned lessons

health check

```
$ dcgmi diag -r 2 -v -d VERB  
Successfully ran diagnostic for group.
```

```
+-----+-----+  
| Diagnostic | Result |  
+=====+=====+  
|  Metadata  |  
| DCGM Version | 3.3.1 |  
| Driver Version Detected | 575.57.08 |  
| GPU Device IDs Detected | 2330,2330,2330,2330,2330,2330,2330 |  
|  Deployment  |  
| Denylist | Pass |  
| NVML Library | Pass |  
| CUDA Main Library | Pass |  
| Permissions and OS Blocks | Pass |  
| Persistence Mode | Pass |  
| Environment Variables | Pass |  
| Page Retirement/Row Remap | Pass |  
| Graphics Processes | Pass |  
| Inforom | Pass |
```

Scaling Cheatsheet

The Ultra-Playbook Cheatsheet

Step 1: Fit model into memory

GPU rich case: 🧑‍💻

- **Small models (<10B):** use a single parallelism technique, e.g. TP or ZeRO-3/DP with Full Recompute across 8 GPUs.
- **Large models (10B+):** requires more than 8 GPUs, you have several options:
 - Combining Tensor Parallelism (TP=8) with Pipeline Parallelism
 - Combining Tensor Parallelism (TP=8) with Data Parallelism (ZeRO-3)
 - Using only ZeRO-3 (i.e. only pure Data Parallelism)
- **512+ GPU scale:** pure DP/ZeRO-3 becomes inefficient due to communication cost - better to then combine DP with either TP or PP
- **1024+ GPU scale,** a recommended setup can be TP=8 with DP (ZeRO-2) and PP
- Special cases: for **long context** consider CP and for **MoE** arch use EP

GPU poor case 🧑‍🔧:

- **Reduce memory:** use full activation checkpointing and/or gradient accumulation

Step 2: Satisfy target global batch size

Experiments tell us which batch size is ideal for training (4-40M tokens). So we either have to increase or decrease the batch size based on step 1 to meet it.

Increase Global Batch Size:

- Scale up DP or CP or gradient accumulation steps

Decrease Global Batch Size:

- Reduce DP or CP in favor of other parallelization strategies

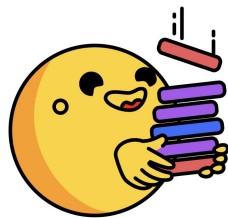
Step 3: Optimizing Training Throughput

There is no general recipe for the best configuration so at this point we should experiment:

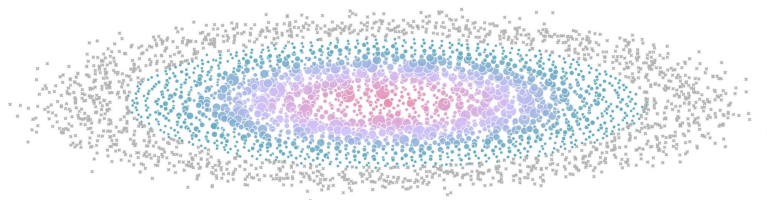
- **Scale up TP** up to the node size to reduce other parallel strategies
- **Increase DP** with ZeRO-3 while keeping target GBS
- **Use PP** if communication becomes a bottleneck for DP
- Play with **micro batch size** to balance max GBS, model size, compute/comms

And much more..

- Nanotron: <https://github.com/huggingface/nanotron>
- TorchTitan: <https://github.com/pytorch/torchtitan>
- Megatron-LM: <https://github.com/NVIDIA/Megatron-LM>
- Ultra-Scale Playbook huggingface.co/spaces/nanotron/ultrascale-playbook
- Smol-Training Playbook huggingface.co/spaces/HuggingFaceTB/smol-training-playbook
- Jax Scaling Book <https://jax-ml.github.io/scaling-book/tpus/>



**The Ultra-Scale Playbook:
Training LLMs on GPU Clusters**



Finally..

As we scale to thousand of GPUs let's be mindful of their energy impact, and try to use them *responsibly*

Thank you 🤗

GitHub/HF Hub/X: [nouamanetazi](#)